

Conxa — Product Requirements Document

Version: 1.0 Status: Foundational Product Definition Owner: Conxa

1. Product Overview

Product Name: Conxa

One-Line Description: Conxa turns any human-performed software workflow into a precompiled, self-healing skill that AI agents can execute reliably — without writing code or touching the target application.

Mission: Make every software platform operable by AI, exactly as humans operate it today.

Vision: A world where AI agents handle repetitive software work end-to-end — not by navigating UIs from scratch on every run, but by executing precompiled, battle-tested skills that already know what to do, where to look, and how to recover when things go wrong. Conxa is the infrastructure that makes that possible.

2. The Problem

AI agents are becoming the default interface for getting work done. But the software they need to operate was built for humans — not for agents.

Today's approaches all break in predictable ways:

- **Traditional RPA** encodes brittle selectors. One UI update and everything breaks. Maintenance costs compound faster than value delivered.
- **Browser automation scripts** are developer tools, not something a SaaS company can hand to a customer.
- **Sending an AI agent to navigate live UI** works for demos. It fails at scale — token costs explode, latency is high, and reliability is inconsistent because the agent rediscovers the interface on every run.
- **Native integrations (APIs, webhooks)** require the SaaS vendor to build and maintain them — expensive, slow, and impossible for long-tail workflows that exist inside a product but don't justify a dedicated API.

The gap: there is no infrastructure layer that lets an AI agent reliably operate existing software at human-equivalent reliability, without requiring the software vendor to build anything.

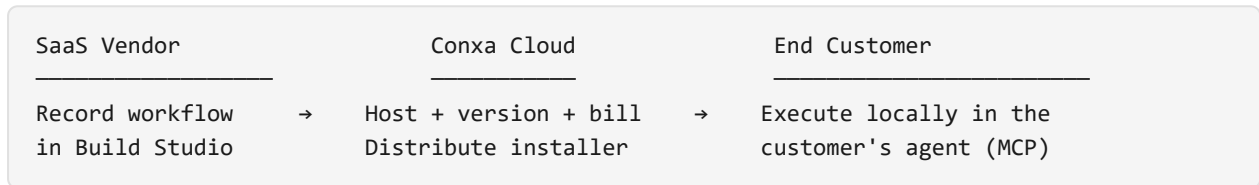
Conxa fills that gap.

3. The Solution

Conxa separates the "teach" step from the "execute" step.

A human performs a workflow once in the **Build Studio**. Conxa records not just the clicks — it captures intent, UI structure, element relationships, visual fingerprints, and recovery context. This session is compiled locally into a **Skill Package**: a structured, versioned execution artifact that encodes everything the runtime needs to execute the workflow reliably.

That Skill Package is published to **Conxa Cloud**, packaged into a branded `.exe` installer, and distributed to end customers. On the customer's machine, the **Conxa Runtime** — a local MCP server — downloads the skill, exposes it as a native tool to whichever AI agent the customer already uses, and executes it with full self-healing recovery. Execution never leaves the customer's machine.



The result: the SaaS vendor teaches the workflow once. Their customers get it forever, always up-to-date, always recoverable.

4. Core Value Proposition

For SaaS companies: Ship AI-native capabilities to your customers without touching your codebase. Record your product's workflows once in the Build Studio — Conxa compiles them into skills and distributes them to your customers as a branded installer. No API, no SDK, no new engineering headcount.

For enterprises: Stop relying on humans for repetitive software work. Give Claude the skills it needs to operate your tools the same way your team does — reliably, at scale, without token waste.

For AI agents: Execute precompiled workflow skills instead of navigating live interfaces from scratch. Lower token cost, deterministic step execution, and built-in recovery that handles UI drift automatically.

5. Why Now

Three shifts are converging:

1. **MCP has become the standard interface between AI agents and tools.** What began as Claude Desktop's protocol is now implemented across the agent ecosystem — coding agents, IDE assistants, CLI agents, and desktop apps from multiple vendors all speak MCP. Conxa is built natively on it, and the runtime registers itself into every major MCP host rather than a single one. A skill recorded once is callable from whichever agent the customer already uses.

2. **AI agents are graduating from demos to production.** Enterprises are now asking how Claude handles their actual software stack — not hypothetically, but operationally. There is no good answer without execution infrastructure.
 3. **SaaS companies need an AI-native distribution channel.** "How do I make my product work with Claude?" is a question every SaaS product team is now asking. Conxa answers it without requiring API investment.
-

6. Target Customers

Primary: SaaS Companies

SaaS vendors who want to make their platform operable by Claude without building native AI integrations.

They record their own product's workflows in the Build Studio, publish skills to the cloud, and distribute them to their customers. Conxa handles compilation, hosting, distribution, versioning, and telemetry.

Relevant verticals: - CRM and sales platforms - Marketing and growth tools - HR and people management platforms - Customer success and support tools - Internal business operations software

Primary: Enterprises

Organizations with high-volume, repetitive software work that currently requires human operators.

They use the Build Studio to record their own internal workflows, deploy skills to their teams, and let their AI agents execute them — through Claude Desktop, a coding agent, or whichever MCP-capable host the team already runs.

Relevant teams: - Operations and back-office - Finance and accounting - Sales operations - Customer support

Secondary

- Automation consultants building skills for client stacks
 - Agencies managing software operations at scale
 - Internal IT teams standardizing software access for AI tooling
 - AI-first startups that need reliable non-API software integrations
-

7. Product Components

Conxa Build Studio

A Windows desktop application (Electron + Python) that runs entirely locally. The SaaS vendor or enterprise user uses it to:

- Record browser workflows using an injected capture bridge
- Review and edit captured workflows step-by-step
- Compile workflows locally into Skill Packages — no cloud involvement in this step
- Build plugin archives and NSIS installer packages
- Publish compiled skills to Conxa Cloud

Recording, compilation, and packaging are entirely local. The cloud is not in the execution path during build.

Conxa Cloud

A thin coordination layer (FastAPI on Render, Next.js on Vercel) that handles:

- Skill package hosting and versioning
- Installer hosting and distribution
- LLM proxy for compile-time AI calls (multi-provider: Groq, Google AI Studio, NVIDIA NIM)
- Billing, plan entitlements, and subscription management (Cashfree)
- Execution telemetry and run analytics
- Team and organization management, role-based access control, and a workspace audit log

The cloud does not record, compile, or execute workflows. It is coordination infrastructure.

Governance. Membership and roles are resolved from the vendor's identity provider, and privileged actions are restricted to workspace owners and admins. Every consequential action — publishing a skill release, changing a plugin's distribution state — is written to an immutable, per-workspace audit log that admins can review in the dashboard. Enterprise buyers treat this as table stakes: they need to answer "who shipped this skill to our customers, and when" without asking Conxa.

Conxa Runtime

A Node.js MCP server that ships inside the vendor's branded `.exe` installer and runs on the end customer's machine. It:

- Registers itself as an MCP server into every AI agent host installed on the machine
- Syncs skill packs from Conxa Cloud (delta sync, SHA-256 verified)
- Exposes skills as native agent tools (`execute_skill` , `list_skills` , `get_skill_inputs` , etc.)
- Executes skills locally via Playwright with a multi-tier self-healing recovery cascade — see `docs/TRD.md §10.1`
- Streams execution telemetry back to Conxa Cloud
- Self-updates by polling the update manifest

Execution never leaves the customer's machine.

Two layers, updated on different clocks. What installs as one program is internally split in two. `conxa-runtime` is the host binary — the heavy, stable layer that carries the browser engine and the OS integrations. It is ~85 MB and changes rarely, on the order of quarterly. `conxa-app` is the logic layer that sits on disk beside it — the executor, the element resolver, the recovery cascade. It is a ~60 KB download and ships with every release.

This split is what makes fixes travel fast. A reliability improvement to the recovery cascade reaches every customer machine as a 60 KB update within a release cycle, rather than requiring an 85 MB reinstall that end users must consent to. Since the vendor's customers — not the vendor — are the ones running the software, an update path that needs their attention is an update path that doesn't happen.

Two safeguards keep that speed from becoming a liability: the host refuses to load an app layer that declares a newer host requirement than it can satisfy, so a fast-moving logic layer can never half-run against an old binary; and a failed update rolls back to the previous version automatically. Both artifacts are integrity-checked, and the manifest describing them is cryptographically signed.

MCP Layer

Conxa Runtime speaks the Model Context Protocol natively, and registers into the agent hosts the customer already has installed rather than a single vendor's app. Supported hosts span desktop assistants (Claude Desktop), coding agents (Claude Code, Codex, Gemini CLI, GitHub Copilot CLI, OpenCode, Goose), IDE-embedded agents (Cursor, VS Code, Windsurf, Zed, Cline, Antigravity, Kiro, Junie), and agent platforms (Factory Droid, OpenHands, Augment, KiloCode, and others) — currently more than twenty, added as the ecosystem grows.

Skills appear to each of these agents as first-class tools. An agent can discover available skills, request required inputs, execute them singly or as a sequence, inspect runtime status, and cancel runs — all through the standard MCP interface, with no custom integration per host.

This is a distribution property, not just a compatibility one: the vendor records a workflow once and it becomes callable from every agent their customers use, without the vendor targeting any of them.

Platform Support

Build Studio is Windows-only. Recording, compilation, and packaging happen on Windows. This is acceptable because the person recording a workflow is the vendor's own product or ops person, not their customer — it constrains one internal seat, not the addressable market.

The Runtime ships for Windows today. A macOS runtime target exists and builds, but macOS is not yet a supported customer platform: there is no macOS installer format, and the host binary and update path are unverified on darwin. Treat macOS runtime support as roadmap, not capability, until an installer ships. This is the single largest platform gap for enterprise deployments with mixed fleets.

8. Key Capabilities

Workflow Recording

The Build Studio injects a capture bridge into the browser that records every interaction at the event level — clicks, inputs, navigation, scroll, focus, iframe transitions, and screenshots. Metadata is captured alongside each action: element role, text content, bounding box, page URL, and frame chain.

Multi-Signal Compilation

Recorded sessions are normalized, deduplicated, and enriched through a local compilation pipeline. Each step is analyzed for:

- **Primary selectors** — multiple compiled strategies ranked by specificity and resilience
- **Semantic intent** — what the step is trying to accomplish, not just where it clicks
- **Anchors** — nearby stable text elements used as spatial landmarks
- **Visual fingerprints** — screenshot crops for vision-based recovery
- **Assertions** — expected post-step UI state to validate correct execution
- **Recovery metadata** — everything needed to re-find and re-execute if the element has moved or changed

Self-Healing Execution — Recovery Cascade

When a step fails to find its target element, the runtime escalates through a multi-tier recovery cascade, only calling Claude once cheaper, zero-token methods are exhausted — see [docs/TRD.md §10.1](#) for the authoritative tier table.

Workflow Editing

After recording, vendors can review the captured workflow step-by-step, modify individual actions, add conditional logic, annotate intent, and verify the execution plan before compilation.

Skill Distribution

Compiled skills are packaged into a self-contained plugin archive and bundled into an NSIS Windows installer. The installer is hosted on Conxa Cloud and linked from the vendor's dashboard. End customers download and run it — the runtime installs itself, registers into every MCP-capable agent host it finds on the machine, and is immediately available in all of them.

Skill Versioning and Sync

When a vendor updates a skill and republishes, the runtime on every customer machine detects the delta at next sync and atomically updates the local skill pack. No customer action required.

Execution Telemetry

Every execution emits structured telemetry — step outcomes, recovery tiers used, latency, failure points. Vendors see this in their Conxa Cloud dashboard as run timelines and aggregate analytics.

9. Major Use Cases

CRM operations — Create leads, update contacts, log calls, manage pipeline stages, generate reports.

Customer onboarding — Provision new accounts, configure initial settings, trigger welcome sequences, set up integrations.

Invoice and finance processing — Upload documents, extract structured data, update accounting systems, trigger approval flows.

HR and people ops — Onboard new employees, provision access, run payroll inputs, generate compliance reports.

DevOps and cloud management — Environment setup, deployment triggers, infrastructure configuration, cloud console operations.

Internal reporting — Pull data from operational tools, populate dashboards, generate and distribute regular reports.

Social and content operations — Schedule and publish content, pull engagement reports, manage accounts across platforms.

10. Competitive Positioning

vs. Traditional RPA (UiPath, Automation Anywhere)

RPA encodes point-in-time selectors into brittle scripts. UI changes break automations. Maintenance is continuous and expensive. These platforms are designed for IT-managed enterprise deployments — not for SaaS vendors to distribute to customers.

Conxa's advantage: multi-signal compilation means skills degrade gracefully rather than breaking hard. Self-healing recovery handles UI drift automatically. And Conxa is distribution infrastructure — skills ship with the product, not as internal IT projects.

vs. Browser Automation Tools (Playwright, Puppeteer)

Developer tools that require code. They have no compilation model, no recovery architecture, no distribution mechanism, and no AI integration. They solve a different problem for a different audience.

Conxa's advantage: no-code skill creation, built-in recovery, and native MCP distribution. A product manager can record a workflow and ship it to customers.

vs. AI Agents Navigating Live UI

Sending an AI agent to navigate a live interface from scratch on every run costs tokens, takes time, and produces inconsistent results because the agent must rediscover the interface continuously.

Conxa's advantage: precompiled skills encode the execution graph once. The agent executes a skill, not a navigation session. Token cost is bounded, latency is lower, and reliability is deterministic.

vs. Native API Integrations

APIs require the software vendor to build and maintain them. They're unavailable for long-tail workflows, require developer resources, and lag behind UI-level capabilities.

Conxa's advantage: zero engineering required from the software vendor. If a human can do it in the browser, Conxa can make it a skill.

11. Business Model

Conxa sells to the vendor or enterprise that *builds* skills, not to the end customers who run them. The buyer is the workspace; their customers install the runtime for free.

What is metered

Pricing follows the two things that actually cost Conxa money and the two that track account value:

Axis	What it limits	Why it's metered
Seats	People in the workspace who can build and publish	Tracks team size and account value
Skill pack slots	Distinct products a workspace can distribute under	Tracks breadth of deployment
Compile credits	Workflow compilations	Compilation is the real LLM cost centre
Human-edit tokens	LLM usage in the workflow editor's assisted-repair paths	The other LLM cost centre, driven by manual fixing

Plan shape

	Free	Starter	Pro	Enterprise
Seats	1	3	10	Negotiated
Skill pack slots	1	3	10	Negotiated
Compile credits / mo	50	300	1,000	Negotiated
Human-edit tokens / mo	1M	10M	50M	Negotiated
Compile LLM pool	Free-tier providers	Paid mid-tier	Paid mid-tier	Highest-quality models

Enterprise carries no defaults — every limit is an explicit contractual override. Subscription billing runs through Cashfree.

What is deliberately *not* metered

Executions are free and unlimited. Skills run entirely on the customer's machine using the customer's own compute — Conxa incurs no marginal cost per run, so charging per execution would price against the product's core promise. This is a structural advantage over every cloud-execution competitor, whose costs scale with customer usage and who must therefore meter it.

The consequence: revenue scales with how many workflows a vendor *builds*, not how hard their customers *use* them. A vendor is never penalised for success, which removes the usual reason customers ration an automation platform.

Cost posture

Compilation is a one-time cost per workflow version; execution is a recurring benefit. The free tier runs on free-tier LLM providers so that trial usage costs effectively nothing, while paid tiers route to higher-quality models where compile quality directly determines skill reliability. Unit economics are modelled in `docs/cost_model.md`.

12. Success Metrics

Product Health

- Skill compilation success rate
- Execution success rate (steps completed without Tier 3+ recovery)
- Recovery success rate (failures resolved by Tier 1–4 before escalation)
- Skill reuse rate across executions

Business Traction

- Active vendor organizations on the platform
- Skills published and distributed
- Active runtimes (customer installs)
- Monthly skill executions
- Enterprise customers under contract

Technical Quality

- Runtime stability (uptime, crash rate)
 - Sync success rate (skill pack delivery)
 - Execution latency (p50/p95 per skill)
 - Self-update success rate
-

13. Product Principles

Reliability over features. A skill that works 99% of the time is worth more than ten features that work 70% of the time. Execution reliability is the core product promise — everything else is secondary.

Teach once, run forever. The human's time is spent once, at recording. Every execution after that should require no human involvement unless something genuinely can't be recovered automatically.

Local execution, cloud coordination. Customer data never transits through Conxa infrastructure during execution. The cloud coordinates — it does not execute. This is a security and trust property, not just an architecture choice.

Zero-cost recovery by default. Tier 1 and 2 recovery cost nothing. LLM escalation is a last resort, not a default fallback. Skills should be compiled with enough redundancy that most real-world UI drift resolves without an LLM call.

AI-native from the protocol up. Conxa is not bolted onto an existing automation platform. It is designed from the ground up for AI agent consumption via MCP — skills are first-class agent tools, not wrapped scripts.

14. Long-Term Vision

Conxa's goal is to become the universal execution layer between AI agents and existing software.

Near-term, this means every SaaS vendor can ship Claude-operable skills alongside their product — turning AI compatibility into a distribution feature, not an engineering project.

Medium-term, this means enterprises running AI workforces where Claude handles entire operational domains — not occasionally, but as the primary operator — with Conxa skills as the execution substrate.

Long-term, this means a marketplace of skills covering the SaaS ecosystem: any agent, any model, any workflow — recorded once by someone, available to everyone. Conxa becomes the npm of AI-executable software operations.

Final Statement

Conxa is not a browser automation tool, a macro recorder, or a traditional RPA platform.

Conxa is execution infrastructure for AI agents — the layer that turns human-performed software workflows into precompiled, self-healing, MCP-native skills that Claude can operate reliably, at scale, on any machine, without touching the target software's codebase.

The interface was already built. Conxa makes it AI-operable.